Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

The purpose of this document is to provide a complete and detailed description of the requirements for **UVSim**, a software simulator designed to help computer science students learn machine language and computer architecture. UVSim will simulate a virtual machine that executes programs written in the BasicML language. This SRS will serve as the foundation for all development, testing, and validation efforts.

1.2 Scope

UVSim is an educational tool that simulates a simple computer architecture including a CPU, accumulator, and main memory (100-word memory). The simulator will interpret and execute BasicML instructions, which include I/O, load/store, arithmetic, and control operations. Key features include:

- **Machine language simulation:** Execute programs written in BasicML.
- **Memory management:** Emulate a 100-word memory where each word is a signed four-digit decimal number.
- **User Interface:** A GUI that provides a clear visualization of the simulation process, input controls, and output displays.
- **Code integration:** A well-structured code base with appropriate layers to support both simulation logic and GUI functionality.

1.3 Definitions, Acronyms, and Abbreviations

- **BasicML:** The machine language that UVSim interprets. Each instruction is a four-digit decimal number with a plus sign (e.g., +2030).
- **UVSim:** The simulator application.
- **Accumulator:** A register in the CPU where arithmetic and logic operations are performed.
- **Word:** A signed four-digit decimal number (e.g., +1234, -5678).
- **GUI:** Graphical User Interface.

1.4 References

- Project proposal documentation.

- Educational materials on machine language and computer architecture.
- Internal design documents and previous prototypes.

1.5 Overview

This SRS document details the functional and non-functional requirements for the UVSim system. It includes descriptions of the overall functionality, specific system features, and design constraints. The document also outlines the user interface requirements and necessary code changes to support a modern, user-friendly GUI.

2. Overall Description

2.1 Product Perspective

UVSim is a standalone educational application that simulates the internal workings of a computer. It is designed to replace or augment traditional teaching methods by allowing students to write and execute machine language programs in a controlled, simulated environment.

2.2 Product Functions

- **Instruction Loading:** Ability to load BasicML programs into memory starting at location 00.
- **Execution of BasicML Instructions:** Support for:
    - **I/O Operations:** READ (10) and WRITE (11).
    - **Load/Store Operations:** LOAD (20) and STORE (21).
    - **Arithmetic Operations:** ADD (30), SUBTRACT (31), DIVIDE (32), MULTIPLY (33).
    - **Control Operations:** BRANCH (40), BRANCHNEG (41), BRANCHZERO (42), HALT (43).
- **Memory Management:** Emulation of 100 memory locations with word-level access.
- **User Interface:** A graphical interface that visualizes the memory, accumulator, CPU, and execution flow.
- **Debugging and Tracing:** Tools to help students step through code execution and understand program flow.

2.3 User Characteristics

- **Primary Users:** Computer science students and educators.
- **Technical Expertise:** Users should have a basic understanding of computer architecture and programming fundamentals.
- **Accessibility:** The interface will be designed to be intuitive for students, with clear labels and workflow guidance.

2.4 Constraints

- **Memory Size:** Fixed at 100 words.
- **Instruction Format:** Each instruction is a signed four-digit decimal number; the first two digits are the opcode, and the last two are the operand.
- **Platform:** Must run on standard desktop operating systems (e.g., Windows, macOS, Linux).

2.5 Assumptions and Dependencies

- Users have basic knowledge of machine language.
- The development environment supports a modular design that allows for separation of the simulation logic and GUI components.
- Dependencies on third-party libraries for GUI development (Tkinter).

3. Specific Requirements

3.1 Functional Requirements

3.1.1 Instruction Set and Execution

- **BasicML Instruction Format:**
    - **I/O:**
        - **READ (10):** Read a word from the keyboard into a specified memory location.
        - **WRITE (11):** Write a word from a specified memory location to the screen.
    - **Load/Store:**
        - **LOAD (20):** Load a word from memory into the accumulator.
        - **STORE (21):** Store the accumulator's value into a memory location.

    - **Arithmetic:**
        - **ADD (30):** Add a word from memory to the accumulator.
        - **SUBTRACT (31):** Subtract a word from memory from the accumulator.
        - **DIVIDE (32):** Divide the accumulator by a word from memory.
        - **MULTIPLY (33):** Multiply the accumulator by a word from memory.
    - **Control:**
        - **BRANCH (40):** Branch to a specified memory location.
        - **BRANCHNEG (41):** Branch if the accumulator is negative.
        - **BRANCHZERO (42):** Branch if the accumulator is zero.
        - **HALT (43):** Terminate program execution.

3.1.2 Memory Management

- **Memory Model:**
    - 100-word main memory.
    - Each memory location can hold either an instruction, data, or remain unused.
- **Word Representation:**
    - A word is a signed four-digit decimal number (e.g., +1234, -5678).

3.1.3 CPU and Register Simulation

- **Accumulator:**
    - Acts as the primary register for arithmetic and logic operations.
- **Instruction Pointer:**
    - Tracks the current execution location in memory.

3.1.4 Program Loading and Execution

- **Program Loader:**
    - Loads a BasicML program into memory starting at location 00.
- **Execution Engine:**

    - Processes each instruction sequentially (unless modified by branch instructions) until HALT is encountered.

- **Error Handling:**
    - Detect and report errors such as invalid instructions, memory overflow, or divide-by-zero operations.

3.1.5 User Interaction and Workflow

- **Input/Output:**
    - Users can input a BasicML program via file upload or direct text entry.
    - Execution outputs (e.g., results of WRITE operations) are displayed on the GUI.
- **Control Flow:**
    - Provide controls for loading programs, running, stepping through instructions, pausing, and resetting the simulation.

3.2 Non-functional Requirements

3.2.1 Performance Requirements

- **Execution Speed:**
    - The simulation should execute instructions with minimal delay, allowing real-time or near-real-time feedback for educational purposes.

3.2.2 Usability

- **User-Friendly GUI:**
    - The interface should be intuitive with clear labels, tooltips, and guided workflows.
    - Provide a visual representation of memory, the accumulator, and the current instruction pointer.
- **Accessibility:**
    - The design should consider accessibility standards to accommodate all users.

3.2.3 Portability

- **Cross-Platform Support:**
    - The application should run on multiple desktop operating systems (Windows, macOS, Linux).

3.2.4 Reliability and Maintainability

- **Robust Error Handling:**
    - The system should gracefully handle errors (e.g., invalid memory access, illegal instruction formats).
- **Modular Architecture:**
    - The code base should be modular to allow for future extensions and ease of maintenance.

3.3 External Interface Requirements

3.3.1 User Interface (GUI)

- **Design:**
    - A GUI will be provided that includes:
        - A main display showing the simulated memory and current state of the CPU (accumulator, instruction pointer).
        - Control buttons for loading, executing, stepping through, and resetting the program.
        - An area for inputting programs (either via file upload or direct text entry).
        - An output area for displaying results and error messages.
- **Wireframe Diagram:**
    - *We need to add here the draft created by Jerica.*

3.3.2 Hardware Interfaces

- **Standard Input/Output Devices:**
    - The system will use the keyboard for input and the monitor for output.

3.3.3 Software Interfaces

- **Operating System:**

- Compatible with Windows, macOS, and Linux.

- **Third-Party Libraries:**
  - Any libraries used for GUI development and other functionalities should be clearly documented.

3.3.4 Communication Interfaces

- **File I/O:**
  - Support for reading program files and writing output logs to disk.

4. System Features

4.1 Machine Language Simulation

- **Instruction Execution:**
  - The engine will parse and execute BasicML instructions, managing control flow and arithmetic operations as described.
- **Memory and Register Simulation:**
  - Simulate a 100-word memory and an accumulator that stores and manipulates data.

4.2 Graphical User Interface (GUI)

- **Wireframe Design:**
  - The GUI will consist of:
    - **Menu Bar:** For file operations (load, save), settings, and help.
    - **Main Panel:**
      - A visual representation of memory (a grid or list of 100 memory cells).
      - A display showing the accumulator and current instruction pointer.
    - **Control Panel:**
      - Buttons for “Load Program,” and “Run”
    - **Output Panel:**

        - Displays program output and diagnostic/error messages.

- **User Interaction:**
    - Users can interact through mouse clicks and keyboard input to control the simulation flow.

4.3 Code Changes and Integration

- **GUI Integration:**
    - Refactor existing simulation logic to decouple the processing engine from the user interface.
    - Ensure that the GUI layer communicates efficiently with the simulation engine, updating displays in real-time.
- **Modular Architecture:**
    - Separate modules for instruction parsing, execution, memory management, and GUI controls to allow independent development and testing.
- **User-Friendliness Enhancements:**
    - Proper labeling of controls.
    - Logical workflow that guides users through program loading, execution, and debugging.
    - Responsive error messages and status updates.